

Scripting for Testing

(With a Spoonful of Perl and a Dollop of Ruby)

Inspired by the test scaffolding idea in Kernighan and Pike's "Practice Of Programming", this article is about generating a regression for a number crunching library.

Every software needs to be tested—and needs to be tested exhaustively. I know, we all practice unit testing and love all that CppUnit/JUnit/TestNG/EasyMock stuff out there.

However, there are other kinds of testing, and it goes by the fancy term, Data Driven Testing. The idea is simple—we arrange the code to work on a piece of data, get the actual result and compare it with the already known, expected result. This is a very powerful idea, as you will see in a minute.

Where do you get this data, though? At times, we need to make it up...

Let's say we need a lot of words to test some string algorithms. Here is one way to generate the list of words...

```
--> perl -le '$k = "a"; print $k++ for (1 .. 1000)'
a
b
...
air
ais
ait
aiu
aiw
aix
-->
```

At times, this simple strategy can answer very complicated questions and validate any assumption you might have about code.

Here is a validation story...

We wanted to use the *tommath* library for number crunching, and though open source products are usually of great quality, we wanted to be very sure we made the right choice.

Now, how do you make sure some complex piece of C++ code is correct without grokking all the code—there being no

time and very little mathematical expertise on hand?

A few years ago, we wanted the number crunching features badly. To be specific, the number crunching had to be correct for numbers with up to 23 digits. In other words, we did not care if you multiplied two numbers, each with 24 (or more) digits in them. However, if the numbers had digits below 24, you needed to be correct.

So we had the code and it did all the math for us, but could we trust it? We wanted to know, badly.

I left home for the day, mulling over the problem, and while travelling, I had an idea... *What about random numbers?* And as usual, Linux is very good at them. So...

```
--> echo "$RANDOM$RANDOM$RANDOM$RANDOM * $RANDOM$RANDOM$RANDOM$RANDOM"
301447958390210712 * 2693820160176179720
-->
```

So far, so good. However, we needed some addition, subtraction and division too. We needed more multiplication and division, as they are more complex to implement and hence, more bug-prone.

A spoonful of Perl

I decided to use the following trick that I first saw in Jon Bentley's *Programming Pearls* -- a book full of the choicest gems.

```
--> perl -lane '$k = rand();
quote> if ($k < .4) { print ""; next }
quote> elsif ($k < .8) { print "f"; next }
quote> elsif ($k < .9) { print "+"; next }
quote> else { print "-"; }
```

We got a handful of * and /, and a few + and -. This goes by the fancy name of 'probability'.

Now the job was simple: we generated a pair of random numbers and pumped them into this Perl script.